University of Oslo

Master's thesis

Master's Thesis

Grover's Algorithm and Quantum Error Correction

Emma Storberg

Computational Science: Physics
60 ECTS study points

Department of Physics
Faculty of Mathematics and Natural Sciences

UiO

Spring 2026

Emma Storberg

Master's Thesis

Grover's Algorithm and Quantum Error Correction

Supervisor:
TBD

Abstract

This thesis studies realistic resource requirements for applying Grover's algorithm to symmetric cryptanalysis under fault-tolerant assumptions. The work investigates how error-correcting architectures impact feasibility, with emphasis on quantum memory and logical operations. Implementations and simulations are developed for small code families, including the 5-qubit and 7-qubit codes, and exploratory analysis is performed for LDPC-style constructions such as surface and BB codes. The results provide practical insight into which design assumptions dominate runtime and overhead in near-term-to-fault-tolerant pathways.

Contents

I Introduction **1**

1 Introduction 3

1.1 Quantum Prerequisities and Notation 5

II Background **7**

2 Grover's Algorithm for Solving Unstructured Search 9

2.1 The Problem Definition 9

2.2 The Algorithm and Circuit 10

2.2.1 Practical Application and Limitations of Grover's Algorithm . . 14

3 Quantum Error Correction 17

3.1 Stable Quantum Memory 17

3.1.1 Toric Surface Code 17

3.1.2 Bivariate Bicycle Code: An Alternative to Toric Surface Code . 20

4 Fault-tolerant Quantum Computation 23

4.1 Universal Gate Sets 23

4.1.1 Choices to Make When Performing Quantum Computations . 27

4.2 Logical Operators in Error Correcting Architecture 28

4.2.1 Implementing Clifford Gates in LDPC Codes 28

4.3 Phase Query Gates in Error Correcting Architecture 28

5 Jaques' Argument 29

III Experiments and Analysis **31**

6 General Limitations when Simulating Quantum Systems. 35

6.1 Proof-of-concept Experiments with Smaller Codes 36

6.1.1 Simulating Toy Version of Grover's Algorithm 37

7 Utilizing Stabilizer Formalism as a Computational Tool 39

7.1 Software Aids 40

7.2 Simulating Stable Quantum Memory with PanQEC 41

7.2.1 Results 41

7.3 Simulating Transversal Logical Operators with PanQEC 41

7.3.1 Results 42

8 The Trouble with the T-gate. 43

8.1 Implemention 43

9 Assessing the Quantum Threat 45

9.1 T-gate Cost as a Bottleneck 45

9.2 Improvements with a Better Error Correcting Code 45

9.3 Grover's Algorithm as a Practical Threat 45

IV Conclusion **47**

10 Conclusion 49

10.1 Future Work 49

10.2 Declaration of AI Usage 49

V Appendices **51**

A Definitions of Smaller Error Correcting Codes Used 53

List of Figures

3.1 A surface code grid with physical data qubits located on the edges, demonstrating a plaquette (Z stabilizer) in blue and a vertex (X stabilizer) in red. Although only one of each is shown here, the entire grid is covered in these stabilizers in a repeating pattern. Image source: Sayedsalehi et al. [19] 18

4.1 Illustration of transversality in a single- and multi-qubit gate. Update image and add source. 24

6.1 The smallest possible surface code consisting of nine data qubits. The indicated eight stabilizers could, in principle, be measured sequentially by a single additional qubit, or measured in parallel by an additional eight qubits. Add source to bibliography. 36

List of Tables

Part I

Introduction

Chapter 1

Introduction

In recent years, quantum algorithms have garnered significant attention for their potential to disrupt traditional cryptographic systems. The security of many important communication channels, including the internet as we know it, is ensured by classical cryptographic protocols, particularly those based on the hardness of certain mathematical problems. With the rise of quantum technology, new algorithms have been found that exploit quantum mechanical principles, such as superposition and entanglement, to solve these problems exponentially faster than classical algorithms. Still, the implementation of quantum algorithms for cryptographic purposes continues to be largely theoretical. A quantum computer capable of breaking the most-used algorithms today is yet to be constructed, as there are many practical limitations to doing so. Thus, practical quantum computers, with thousands of qubits and low error rates, remain an area of active research; there is considerable anticipation surrounding the eventual realization of the theoretical promises associated with quantum computing. As we move into the post-quantum era, it will become increasingly important to assess our cryptographic protocols in light of the most recent developments in the field, in order to maintain and protect critical infrastructure and assets.

Brute-force search is a technique that may be employed to break cryptographic systems that use so-called *symmetric* keys, which means the the same keys for encryption and decryption [1], as opposed to *asymmetric* algorithms, which use separate encryption and decryption keys. Performing such a search involves quite simply checking every single possible key option until the right key is found, an inordinately time-consuming process if the key space is large enough. Our current protocols ensure security against these searches by having key lengths large enough that it is considered computationally infeasible for a classical computer to identify one in any reasonable amount of time. A closer look at the popular symmetric-key algorithm *Advanced Encryption Standard* (AES) [2] exemplifies this: It exists in versions that use key sizes of 128, 192 and 256 bits, resulting in search spaces of 2^{128}, 2^{192} and 2^{256} possible keys respectively.

A relevant quantum algorithm here is *Grover's algorithm* [3], in which a procedure is given for a quantum computer to perform a search much faster than brute-force search can be done on classical computers, thereby potentially threatening the security of AES. As we will see, Grover's algorithm offers a quadratic improvement over classical algorithms, meaning the number of operations needed when we use this quantum algorithm is on the order of the square-root of the number of operations required when solving the same problem classically. However, finding the exact runtime is not as straightforward as the asymptotic estimates, which do not consider constant values, nor do they take into account what kind of parallelization may be possible. Analyzing the true cost of using Grover's algorithm in reality is a nontrivial problem that warrants further exploration, which constitutes the core of this project.

Building a quantum machine in practice is no easy feat. The root of the issue is that quantum information is extremely fragile. The qubits that store information are unstable and very vulnerable to noise that changes the value the qubit holds. We therefore wish to construct circuits of multiple qubits that work together to protect the information from errors, such that they form a single logical qubit that can have its errors located and corrected, making them more stable. This approach, though useful, will often require large numbers of qubits, which poses an engineering challenge to keep them all as stable as possible, facilitate connections between them, and perform computations on the logical qubits, rather than on the physical qubits individually.

Jaques considers this in his 2024 analysis of Grover's algorithm as a quantum attack on AES [4], where he suggests that due to the error correction models and gate sets currently in widespread use, a physical implementation of Grover's algorithm will never see the light of day on anything resembling current quantum architectures (or maybe ever). This project uses his argumentation as a starting point for further analysis, aiming to tackle some of the assumptions he makes and explore alternatives to some of the runtime bottlenecks he identifies.

I am planning to add a little more info towards the end here about my experiments and method as soon as I make a final decision on the direction I want to take this. It will probably include discussions of other error correcting codes like *BB codes*. As in, "since Jacques' argumentation bases itself so much on our continued usage of the surface code, what does his analysis look like under the assumption of a better error correcting code?"

Add a brief, non-technical note that classical simulation of quantum systems is intractable beyond small scale, motivating why this is hard to study experimentally, i.e. justifying my method.

A major problem with quantum computing research is that it is problematic to simulate quantum systems classically beyond small, trivial base cases; indeed, if we could simulate

quantum systems without issue in general it would beg the question why we need to realize them at all. This creates a limitation for us in studying Grover's algorithm, because it is hard to take circuit diagrams and see how it will work in practice. This project explores Grover's algorithm and what it can be used for to threaten cryptosystems and the security afforded by AES, what we would need to build it in practice and why it's so hard, why simulating the quantum hardware is equally as hard or harder, and covers the use of some methods and software we can use to research and simulate quantum systems.

1.1 Quantum Prerequisities and Notation

We assume you know basic Dirac notation, maybe say that n is the number of qubits and N is 2^n?

Hadamard gate, superposition

Pauli matrices

The probability of measuring a particular component of a state vector in superposition is its complex scalar squared: "As we know, in a quantum state vector, the amplitude of any component squared represents the probability of seeing that component after measurement."

Part II

Background

Chapter 2

Grover's Algorithm for Solving Unstructured Search

Chapter 2 is adapted from parts of my previous work [5], with revisions and additions for clarity.

2.1 The Problem Definition

Formally, the computational problem that Grover's algorithm addresses is a quite general problem known as *unstructured search* [6]. It can be described as follows:

The unstructured search problem

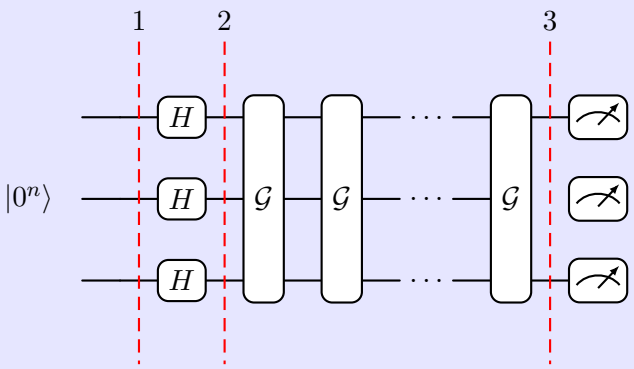
Let Σ denote the binary alphabet $\{0, 1\}$. Suppose we are given a function $f : \Sigma^n \to \Sigma$ that we can compute efficiently. We wish to determine the binary strings $x \in \Sigma^n$ such that $f(x) = 1$, or "no solution" if no such x exists.

We call this search problem *unstructured* because f can be an arbitrary function, meaning there is no underlying pattern or constraint, often called a *promise* [6], that we can take advantage of to systematically search for a solution string x, if one even exists. This means that in the classical approach to solving this problem, we can be sure of the solution only after checking each of the 2^n possible strings, thus requiring 2^n queries.

2.2 The Algorithm and Circuit

Circuit diagram of Grover's algorithm

The algorithm essentially consists of three main parts: we have initialization of n qubits into an equal superposition (1), followed by t Grover iterates (2), and finally measurement to reveal a candidate solution (3). These three stages are displayed and labeled in the circuit diagram below:



Let's dive into the algorithm. Our first step is using the *Hadamard gate* (denoted by H) [7] on each of the n input qubits. H has no classical equivalent, and using it allows us to take advantage of a strictly quantum phenomenon known as *superposition* [8]. This means the qubit can evaluate to either 0 or 1, and we can apply processing to it as if the qubit holds both values simultaneously. Thus, in finding a binary string of length n, using the Hadamard gate on $|0^n\rangle$ lets us evaluate all 2^n strings in Σ^n concurrently. For convenience, we will use the letter N to refer to the value 2^n from this point on.

Next is the Grover iteration step, represented by sequential application of the Grover operator $\mathcal{G}$ in the diagram. Its definition and circuit diagram are explained in detail on the next page.

Should I reference this more formally?

The Grover operation $\mathcal{G}$

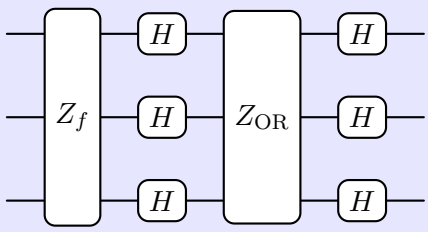
The gate sequence needed for the Grover operation $\mathcal{G}$ is:

$$H^{\otimes n} Z_{\mathrm{OR}} H^{\otimes n} Z_f$$

where Z_f is the phase query gate for the function f, and Z_{OR} is the phase query gate for the n-bit logical OR function. They are defined as follows:

$$Z_f : |x\rangle \mapsto (-1)^{f(x)} |x\rangle \qquad \forall x \in \Sigma^n$$
$$Z_{\mathrm{OR}} : \begin{cases} |x\rangle & x = 0^n \\ -|x\rangle & x \neq 0^n \end{cases}$$

Since the gates are applied from right to left, we apply Z_f first, followed by H, then Z_{OR}, and finally another set of Hadamard gates. The circuit diagram of the Grover operation on three bits looks like this:



Concerning evaluation of the circuit, only the Z_f gate requires a query to f. Thus, the number of iterations t we perform with $\mathcal{G}$ corresponds to how many queries to f we need.

A natural way to use the algorithm for search problems is to choose a number of iterations t of the Grover operation, run the circuit to obtain a candidate solution x, and check if x is a solution (in this case whether $f(x) = 1$). If so, we return x, otherwise we run the algorithm again (possibly with a different value for t) or return "no solution". This sounds simple enough, but still, important questions remain unanswered: What is the "suitable" number of operations t, how do we identify it, and why must it be chosen so precisely?

To understand what is happening here, it is helpful to separate the possible solution strings in Σ^n into two categories, which we will call solutions and nonsolutions. These sets can be described as follows:

$$A_0 = \{x \in \Sigma^n : f(x) = 0\}$$
$$A_1 = \{x \in \Sigma^n : f(x) = 1\}$$

where A_0 contains all the strings that evaluate to 0 when given as input to f, and A_1 has the strings we seek that evaluate to 1. In particular, we are interested in uniform superpositions over these two sets[1], defined as:

$$|A_0\rangle = \frac{1}{\sqrt{|A_0|}} \sum_{x \in A_0} |x\rangle$$
$$|A_1\rangle = \frac{1}{\sqrt{|A_1|}} \sum_{x \in A_1} |x\rangle$$

An important observation here is the fact that we can write the initialized uniform superposition as a linear combination of the states $|A_0\rangle$ and $|A_1\rangle$:

$$H^{\otimes n} |0^n\rangle = \sqrt{\frac{|A_0|}{N}} |A_0\rangle + \sqrt{\frac{|A_1|}{N}} |A_1\rangle$$

The key idea we utilize is this: **The state of the qubit register remains in the subspace spanned by $|A_0\rangle$ and $|A_1\rangle$ after every application of $\mathcal{G}$.** Thus, we would like to process the initialized state $H^{\otimes n} |0^n\rangle$ (with uniform amplitude across all N components) such that through successive iterations, we nudge the amplitude of the solution component $|A_1\rangle$ slightly higher, while lowering the amplitude of the nonsolutions $|A_0\rangle$. As we know, in a quantum state vector, the amplitude of any component squared represents the probability of seeing that component after measurement. Provided we find the suitable number of iterations t, we can push the amplitude of the state corresponding to the solutions arbitrarily high, and consequently see a high probability that this state is measured.

We can show that by applying $\mathcal{G}$ on the $|A_0\rangle$ and $|A_1\rangle$, we obtain the following linear combinations [6]:

$$\mathcal{G} |A_0\rangle = \frac{|A_0| - |A_1|}{N} |A_0\rangle + \frac{2\sqrt{|A_0| \cdot |A_1|}}{N} |A_1\rangle$$
$$\mathcal{G} |A_1\rangle = -\frac{2\sqrt{|A_0| \cdot |A_1|}}{N} |A_0\rangle + \frac{|A_0| - |A_1|}{N} |A_1\rangle$$

This clearly demonstrates that we are still in the span of $|A_0\rangle$ and $|A_1\rangle$ after an application of $\mathcal{G}$. Moreover, we can describe the action of $\mathcal{G}$ as a rotation matrix, and by once again following the example of Qiskit [6], we can also rewrite it as the square of a somewhat simpler-looking matrix:

$$\begin{bmatrix} \frac{|A_0|-|A_1|}{N} & -\frac{2\sqrt{|A_0|\cdot|A_1|}}{N} \\ \frac{2\sqrt{|A_0|\cdot|A_1|}}{N} & \frac{|A_0|-|A_1|}{N} \end{bmatrix} = \begin{bmatrix} \sqrt{\frac{|A_0|}{N}} & -\sqrt{\frac{|A_1|}{N}} \\ \sqrt{\frac{|A_1|}{N}} & \sqrt{\frac{|A_0|}{N}} \end{bmatrix}^2$$

[1] Importantly, we will assume from here on that A_0 and A_1 are both nonempty. However, we can still easily see what happens in the last two cases (all strings evaluate to 0, or all strings are solutions), as the former will return no solution, and in the latter case, any string is a solution. Since we are able to handle these special cases, we can safely continue our analysis with this assumption.

Now it is easier to see the similarity to the general form of a rotation matrix, with $\theta = \sin^{-1}\left(\sqrt{\frac{|A_1|}{N}}\right)$:

$$\begin{bmatrix} \sqrt{\frac{|A_0|}{N}} & -\sqrt{\frac{|A_1|}{N}} \\ \sqrt{\frac{|A_1|}{N}} & \sqrt{\frac{|A_0|}{N}} \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Since squaring a rotation is the same thing as applying it twice, we return to the original rotation matrix for $\mathcal{G}$, and write it as:

$$\begin{bmatrix} \frac{|A_0|-|A_1|}{N} & -\frac{2\sqrt{|A_0|\cdot|A_1|}}{N} \\ \frac{2\sqrt{|A_0|\cdot|A_1|}}{N} & \frac{|A_0|-|A_1|}{N} \end{bmatrix} = \begin{bmatrix} \cos 2\theta & -\sin 2\theta \\ \sin 2\theta & \cos 2\theta \end{bmatrix}$$

with θ defined as before. Thus, we have found a rotation matrix that describes the action of $\mathcal{G}$ on the subspace spanned by A_0 and A_1. This also means that the state of the qubit register after initialization can be described as:

$$H^{\otimes n}\,|0^n\rangle = \sqrt{\frac{|A_0|}{N}}\,|A_0\rangle + \sqrt{\frac{|A_1|}{N}}\,|A_1\rangle = \cos\theta\,|A_0\rangle + \sin\theta\,|A_1\rangle$$

So, each time we apply $\mathcal{G}$, the state of the register is rotated by 2θ, and we can find a closed-form expression for the state after t applications of $\mathcal{G}$. For brevity, we will henceforth refer to the initialized qubit register $H^{\otimes n}\,|0^n\rangle$ as $|u\rangle$.

$$\mathcal{G}^t\,|u\rangle = \cos((2t+1)\theta)\,|A_0\rangle + \sin((2t+1)\theta)\,|A_1\rangle$$

This expression makes sense intuitively: 2θ for each application of $\mathcal{G}$, and one additional θ to account for the state we started in.

Now recall that for any quantum state of the form $\alpha\,|A_0\rangle + \beta\,|A_1\rangle$, we measure a solution $x \in A_1$ with probability $|\beta|^2$. With the state in question $\mathcal{G}^t\,|u\rangle = \cos((2t+1)\theta)\,|A_0\rangle + \sin((2t+1)\theta)\,|A_1\rangle$, the probability of measuring a solution after t iterations is $\sin^2((2t+1)\theta)$. This is the probability we want to maximize in order to find a solution to the search problem with Grover's algorithm, and as such, we call $|A_1\rangle$ our *target state* [6].

Another consideration is that we would also like to minimize t, as this is the number of queries to f we have to make. Thus, to reach our target state with a probability close to 1 while minimizing t, we aim to find t such that $2(t+1)\theta \approx \frac{\pi}{2}$, since this is the smallest angle where the square of the sine function equals 1. In terms of t, we aim for $t \approx \frac{\pi}{4\theta} - \frac{1}{2}$. We cannot guarantee that this value of t is an integer, so we find the number of iterations t by choosing $t = \lfloor \frac{\pi}{4\theta} \rfloor$. For the algorithm to work properly, it is also important that we do not iterate past the recommended iteration number, lest we avoid "overshooting" our solution by rotating past it. If we simply continue applying $\mathcal{G}$, we will in fact decrease the probability that the solution state is measured, and over time, we will oscillate between solutions and nonsolutions as we rotate the register state around the unit circle [6].

Up to this point, we have studied a general version of Grover's algorithm that can be applied to a number of search problems, ranging from those we know to have a single solution, a larger solution set of a fixed size, or even an unknown number of solutions. We have determined that an iteration number of $t = \lfloor \frac{\pi}{4\theta} \rfloor$ (and no more) will maximize the probability of finding one of these solutions, but what we must also consider is the fact that this way of setting iteration number t also depends on how many solutions there are, as θ depends on the size of the set A_1. Fortunately, since we are considering only the cryptographic application of using the algorithm to search for a single key from some symmetric cryptographic protocol, we can restrict ourselves to the cases where we know there is exactly one solution (or one cryptographic key).

Unique search, as this problem is formally known, is equivalent to the general unstructured search problem, with the additional promise that there is exactly one solution string x. Recall that in the general case, $\theta = \sin^{-1}\left(\sqrt{\frac{|A_1|}{N}}\right)$. Now that we have determined there is only a single solution, we know that $|A_1| = 1$. Thus, for large N, we see that $\theta = \sin^{-1}\left(\sqrt{\frac{1}{N}}\right) \approx \sqrt{\frac{1}{N}}$, since for smaller angles, the sine function starts to look more like the identity function [6]. If we use this approximation of θ in our previously determined expression $t = \lfloor \frac{\pi}{4\theta} \rfloor$, we get $t = \lfloor \frac{\pi}{4}\sqrt{N} \rfloor$. Since t represents queries to the function f, we see the quadratic improvement mentioned earlier: Rather than N queries, we are now on the order of the square root of that.

2.2.1 Practical Application and Limitations of Grover's Algorithm

At first glance, the improvements afforded by Grover's algorithm seem substantial. Though not a perfect algorithm in the sense that there is a probabilistic element and some chance of error[2], Grover's clearly has an advantage in runtime complexity over the classical strategy. Therefore, we identify that there is some trade-off between computational efficiency and certainty of outcome when choosing to use Grover's to tackle the unstructured search problem. In terms of correctness, it can be proven analytically that probability of success when using Grover's is always greater than or equal to $1 - \frac{1}{N}$ [6], meaning success is typically likely, particularly as N grows large. We are also free to run the algorithm in its entirety as many times as we choose, and if any one of them succeeds, it will identify a solution, with a high chance of doing so using fewer queries to f than its classical counterpart. With the classical cost on the order of 2^{128} for AES-128, a naive calculation finds the new cost with Grover's square-root speedup to be around 2^{64}, perhaps with some small constant to account for overhead and setting up the quantum circuit. However, this estimation does not take into account a number of practical concerns and considerations.

First, Grover's algorithm, as described in Section 2.2, uses an oracle f, which in our case

[2] in contrast to the classical runtime we discussed, where the exhaustive search strategy leaves no chance of error whatsoever

corresponds to the encryption/decryption algorithm. AES, a complicated algorithm in its own right, must be implemented in the quantum hardware using computationally expensive gates and structures [4, 9], which we introduce and discuss in more detail in Section 4.1. This adds significant cost to the runtime of the algorithm [4], which the commonly cited Big O abstraction does not capture [10]. Exploring circuit designs of AES and their costs is outside of the scope of this project, but it is an active field of research [11, 12, 9] that will be crucial for full asymptotic analysis and practical implementation of Grover's algorithm applied to AES.

which is it?

Secondly, as pointed out by Jaques, any realistic attack will require parallelization [4], and unfortunately, Grover's algorithm does not parallelize well. If we consider the full key space of N keys and partition it into P subsets, we can assign each of the P partitions to a machine responsible for a search space containing $\frac{N}{P}$ keys. With the square-root speedup over the classical method as discussed, we can now find the key in $\mathcal{O}(\sqrt{\frac{N}{P}})$ time. When comparing this to serial Grover's with a runtime of $\mathcal{O}(\sqrt{N})$, we find that parallelizing only reduces the runtime by a factor of $\sqrt{P}$, not P. Worse yet, the additional partitions add an overhead, with the total cost of operations going up to $P \cdot \mathcal{O}(\sqrt{\frac{N}{P}}) = \mathcal{O}(\sqrt{P \cdot N})$. Thus, in our case with AES-128, the bad parallelism of Grover means its cost is certainly higher than the naive estimation of 2^{64}.

The parallelization argument holds for any algorithm on the order of $\sqrt{N}$, and it turns out $\mathcal{O}(\sqrt{N})$ is the best we can do: For the general unstructured search problem, it has been proven that Grover's algorithm is *asymptotically optimal* [13, 14], meaning that as the search space grows, it is not possible to find an algorithm that performs better than on the order of $\sqrt{N}$ in the worst case. Despite its clear limitations, Grover's is thus still a contender to break the security of AES and similar symmetric encryption schemes by quantum computing, motivating the exploration we undertake in this project.

Chapter 3

Quantum Error Correction

3.1 Stable Quantum Memory

As we briefly touched upon, future quantum computers will certainly require some form of error correction, as quantum systems are prone to noise, causing logical errors during computation. Unlike in a classical computer where the only errors we consider are bit-flip errors (where 0 is interpreted as 1, or vice versa), quantum information can be destroyed by two separate types of errors: bit-flips and phase-flips. Having two types of fundamental errors to correct, in combination with quantum hardware being inherently more difficult to work with from an engineering perspective, has the unfortunate result of massively complicating the circuitry required for computation as soon as any practically viable error correction measures are introduced, a topic we will revisit in more detail in Section 4.1.

Justify why these types of errors span all errors

3.1.1 Toric Surface Code

The standard method of error correction that is used in all existing quantum computers built for fault-tolerance is the *surface code*. Following the discovery of the toric code[1] in 1997 [16], toric surface code has remained a prevalent error correcting architecture because it is based on simple principles and maintains geometric locality[2].

This is my impression, but it's hard to find a specific source.

In essence, surface code works by creating a grid of data qubits and measurement qubits interspersed and defining *stabilizers* that flag bit-flip (X) errors and phase-flip (Z) errors

[1] There is some inconsistency in the literature about what constitutes a *toric code*, a *surface code*, and a *toric surface code*. We adhere to the nomenclature set by IBM [15], where toric codes exist in a grid across the surface of a torus with periodic boundary conditions, and surface codes and toric surface codes are used interchangeably to refer specifically to the portion of a torus grid laid flat with non-periodic boundary conditions.

[2] The only links required in the surface code are between neighboring qubits, as opposed to long-distance connections required by certain other codes that are more challenging from an engineering standpoint.

when they occur in specific locations. The error correcting capability lies in measuring the stabilizers across the grid repeatedly, decoding where errors have occured, and applying the appropriate correction based on the stabilizer measurement. The stabilizer formalism can in many ways be viewed as the quantum generalization of parity checks [17, 18], which are often used similarly in classical error correction to identify where and how to restore corrupted data.

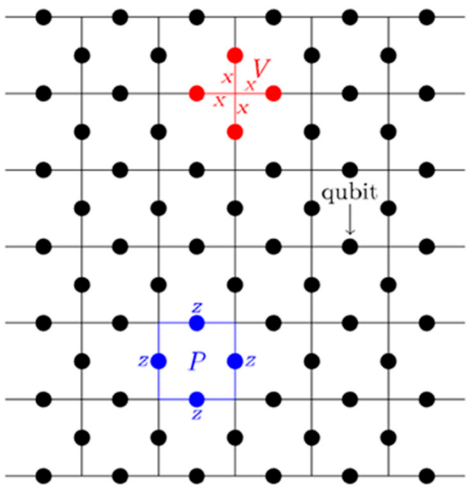


Figure 3.1: A surface code grid with physical data qubits located on the edges, demonstrating a plaquette (Z stabilizer) in blue and a vertex (X stabilizer) in red. Although only one of each is shown here, the entire grid is covered in these stabilizers in a repeating pattern.
Image source: Sayedsalehi et al. [19]

I might remake this graphic myself.

In Fig. 3.1, we show how taking four qubits surrounding a vertex or a square face in the grid (known as a *plaquette*) defines an X or Z stabilizer respectively. Check qubits for measurements in the stabilizers are a crucial element of quantum error correction specifically, as measuring the actual state of data qubits to identify data corruption would destroy their states, and thus the data itself.

The stabilizer formalism allows us to define algebraically the units described visually in Fig. 3.1 with two sets of commuting stabilizer generators. For each vertex v in the grid, we have an X stabilizer

$$S_v^X = \prod_{j \in \text{star}(v)} X_j,$$

which multiplies the Pauli X operator on the four[3] edges touching v. We similarly define a Z stabilizer for each plaquette p as

$$S_p^Z = \prod_{j \in \text{boundary}(p)} Z_j,$$

which multiplies the Pauli Z operator on the edges surrounding p. The groups formed by all S^X and S^Z respectively have elements that are all tensor products of Pauli operators

[3] or fewer at the grid boundaries

and that commute. Together, each of these groups has a shared eigenspace constrained by

$$S_i \left|\psi\right\rangle = + \left|\psi\right\rangle \qquad \forall i,$$

where S_i represents a general stabilizer (either X or Z) applied to an appropriate vertex or plaquette, denoted by i. This eigenspace contains the legal code words, or in this case, the states corresponding to logical 0 and logical 1 (henceforth denoted by $\left|0\right\rangle_L$ and $\left|1\right\rangle_L$ respectively).

The Pauli operators satisfy the anticommutation relation $\sigma_j\sigma_k + \sigma_k\sigma_j = 2\delta_{jk}\mathbb{I}_2$, where σ_i are the Pauli operators according to the naming convention $\sigma_1 = X, \sigma_2 = Y, \sigma_3 = Z$, δ_{jk} is the Kronecker delta and $\mathbb{I}_2$ is the 2×2 identity matrix [20]. Thus, if an error operator E commutes with a stabilizer S_i, the eigenvalue of the surrounding qubits' state is unchanged:

$$S_i(E\left|\psi\right\rangle) = E(S_i\left|\psi\right\rangle) = +E\left|\psi\right\rangle.$$

However, if E and S_i anticommute, we will have $S_i(E\left|\psi\right\rangle) = -E(S_i\left|\psi\right\rangle)$ instead, which changes the eigenvalue of the local stabilizer state from $+1$ to -1.

At this point, we can do a measurement of the ancillary check qubits of the stabilizers to determine which ones violate our constraints and flag errors in their neighbors. Thinking back to the notion of parity checks, we consider the result of each stabilizer measurements a remapping of the parity check values 0 and 1 to $+1$ and -1 respectively. This is a very convenient mapping for us, because when we work with Z operators[4], ± 1 happen to be the eigenvalues of the eigenstates $\left|0\right\rangle$ and $\left|1\right\rangle$. Thus, ± 1 represent a way for us to access information about the state without measuring the state itself. In other words, if we measure the check qubits in the stabilizers and see $\left|0\right\rangle$, we know the data qubits are in the eigenstate $+1$, and we conclude no error has occurred among the neighboring qubits of this stabilizer[5]. However, if we see $\left|1\right\rangle$, we know the surrounding data qubits are in the eigenstate -1, and an error has occurred here. Together, the eigenvalues s_i that we observe at each of the m stabilizer measurements form a string $(s_1, s_2, \ldots, s_{m-1}, s_m) \in \{\pm 1\}^m$, called the *syndrome* of the full state. It tells us about what errors are likely to have occurred and where, which we can use to determine what corrections we would like to apply to restore the logical state.

While conceptually simple, the toric surface code has a clear drawback in that it is very qubit-hungry, with estimates predicting a ratio around a 1000 to 1 of physical to logical qubits required for realistic error correction rates [4, 21]. This has sparked further study and innovation in order to uncover more favorable schemes in terms of resource efficiency.

[4] A measurement of X is slightly more involved, but still unproblematic. The relation $HXH = Z$ allows us to simply apply H and do a measurement of Z instead.

[5] Strictly speaking, we only know an even number of errors has occurred. For instance, two errors in the neighboring qubits of the stabilizer will also flip its state back into the eigenstate $+1$, and thus be undetectable.

3.1.2 Bivariate Bicycle Code: An Alternative to Toric Surface Code

The continued usage of the surface code at the current correction rates is one of the underlying assumptions in Jaques' predictions. Thus, it is interesting for the sake of our analysis to see how the prognosis for Grover's algorithm changes with a different choice of error correcting code. A few promising examples can be found in the *Low Density Parity Check* (LDPC) family of codes, of which the surface code is a special case.

Explain and introduce LDPC codes properly.

In particular, *Bivariate Bicycle code*, also known as BB code, is a strong potential candidate [22]. Though similar to surface codes at a high level of abstraction, BB codes differ in that their stabilizers are not geometrically local, and has therefore been called "surface code with extra long-distance checks" [23]. These checks allow for a number of advantages over the surface code, such as a higher threshold for error correction and lower qubit overhead [22].

We begin by defining the following shift matrices:

$$x = P_\ell \otimes \mathbb{I}_m, \quad y = \mathbb{I}_\ell \otimes P_m,$$

Can I use m here when I used m to mean something else previously? I'm running out of letters!

where $\mathbb{I}_m$ is the $m \times m$ identity matrix and P is a cyclic permutation matrix in the form $\sum_{i=1}^{\ell} |i\rangle \langle i+1|$, which maps $|i+1\rangle$ to $|i\rangle$ [23]. In addition, x and y satisfy $xy = yx$ and $x^\ell = y^m = \mathbb{I}_{\ell \times m}$. We use x and y to represent stepping through the grid in the x- and y-direction respectively.

Formally, BB code is defined as a pair of $\ell m \times \ell m$ binary matrices $A(x, y), B(x, y)$ that are sums of our shift matrices x and y to integer powers. These matrix polynomials are then used to define parity check matrices H^X and H^Z:

$$H^X = [A|B], \quad H^Z = [B^\intercal|A^\intercal].$$

The dimension of H^X and H^Z is thus $\ell m \times 2\ell m$. For our purposes, $2\ell m = n$, the number of physical qubits, and $\ell m = \frac{n}{2}$, which is the number of vertex and plaquette stabilizers in the grid. We use these two matrices to index which qubits in the grid are to be acted on by stabilizers. Each row contains the qubit indices for a specific stabilizer, and each column is a binary indication of whether a specific qubit in the grid is part of that stabilizer. For instance, in BB code, each row will therefore have six 1s, as each stabilizer acts on six qubits, while the surface code in this notation would have four 1s per row (at most), because each stabilizer acts on only four qubits. The shifting mechanism defined by A and B is the mathematical way to describe the systematic construction of an evenly-spaced pattern repeating throughout the grid. With some appropriate indexing of the grid, we are able to specify the exact qubits to include in each stabilizer, uniquely defining a BB code.

Measuring a stabilizer corresponds to checking whether the associated parity condition set in the H^X and H^Z matrices is satisfied. Specifically, we require that

$$H^X (H^Z)^\intercal = 0.$$

This requirement ensures that the stabilizers commute, so that we can apply similar logic as with the standard surface code to identify the stabilizers for which this constraint does not hold, where the anticommutation relation will let us detect stabilizers in the -1 eigenstate. The syndrome that is produced informs the corrections we make as a result.

It is common to use the notation $[[n, k, d]]$ to specify the parameters of an error correcting code, where n is the number of physical qubits, k is the number of logical qubits we encode, and d is the number of errors the code can correct. IBM, the developer of the BB code, has primarily considered a BB code they call the *gross* code[6], which is a [[144, 12, 12]]-code (though many other variants exist and are perhaps better suited for our purposes — see **??**). In this specification, 144 physical qubits are required to encode 12 qubits, and the code can correct up to 12 errors without issue [22]. With the 1000 to 1 ratio needed for the toric surface code, the gross code (and BB codes in general) offer a clear improvement in this regard. As an additional plus, the connections between qubits in the BB code form a Tanner graph [24], which is a bipartite graph that can be separated into two disjoint sets. In this structure, nodes (qubits) within each set are only connected to members of the other, never to one another. This can be useful when physically constructing this architecture, since it allows the hardware to be split cleanly into two separate levels.

[6] after the term for a dozen dozen, known as a gross

Chapter 4

Fault-tolerant Quantum Computation

4.1 Universal Gate Sets

Assuming we have created some scheme for fault-tolerant storage of quantum data, we still face a number of significant challenges before we can perform any fault-tolerant computations. In particular, the *Eastin-Knill theorem* [25, 26] is commonly paraphrased as follows:

The Eastin-Knill theorem (paraphrased[1])

No quantum error correcting code can possess a universal set of transversal logical gates.

The full proof of this theorem, given by Eastin and Knill in their paper [25], relies on group theory and the algebraic properties of the error correcting codes themselves, and is far more rigorous and technical than we need for this project. For our purposes, it is enough to understand that the theorem places restrictions on the fault-tolerant quantum computation architecture. So far, we have not seen the terms *transversal* and *universal*, but in this chapter, we will define them and argue why they are useful properties for a gate and gate set to have respectively, and thus why the Eastin-Knill theorem is such an important result in quantum computing.

Before we can fully understand what the theorem is saying, though, we need to to delve further into how applying gates to quantum states forms a circuit. For this, we need the concept of a *unitary matrix*, or simply a unitary, which is a norm-preserving matrix $U \in \mathbb{C}^N \times \mathbb{C}^N, \;\; N \in \mathbb{N}$, whose conjugate transpose $U^\dagger$ is also its inverse, meaning

$U^\dagger U = UU^\dagger = \mathbb{I}_N$. Mathematically, we can represent any quantum circuit on n qubits with a single unitary matrix.

$$U \in \mathbb{C}^{2^n \times 2^n}.$$

While this representation can certainly be useful, like in classical simulations with very few qubits, collapsing the circuit into one matrix in this way generally does not capture a lot of important behaviors and phenomena we are interested in when studying quantum error correction. In hardware, we never apply a single giant unitary gate; instead, as we have seen in the description of Grover's algorithm in Section 2.2, we decompose the computation into smaller, simpler gates. In doing so, we could in principle have our pick from a large selection of possible gates, but if for simplicity we restrict ourselves to implementing only a few different types, we would like the set of gates we choose to be *universal*. A universal gate set can approximate any such unitary U to an arbitrary degree of precision with a finite sequence of gates, an important property for our gate set to have, as without it, there will always be computations we cannot perform.

The Eastin-Knill theorem tells us that we cannot have a universal gate set in which every gate is *transversal*, where transversality refers to the property that a gate that can transform a logical qubit by applying a gate to each of the n physical qubits encoding it in a specific error correcting code. The physical qubits encoding a single logical qubit are collectively called a *code block* or *error-correcting patch*, which we will use interchangeably. A gate acting on k logical qubits therefore acts across k such blocks. A transversal gate can be expressed like this in a particular error correcting code:

$$G_L = \bigotimes_{i=1}^{n} G_i^{(1,2,\ldots,k)}.$$

where $G_i^{(1,2,\ldots,k)}$ acts only on the i-th qubit in each of the k blocks, with no coupling between positions $i \neq j$ within any single block. The transversal property is illustrated for a single- and multi-qubit gate in Fig. 4.1.

Figure 4.1: Illustration of transversality in a single- and multi-qubit gate. Update image and add source.

As this illustration shows, transversal gates ensure single-qubit errors do not propagate to multiple qubits within an error-correcting patch, making them inherently fault-tolerant: A single error occurring when applying the gate will produce at most one error for the error-correcting patch to correct. This is clearly a desirable property for gates in our chosen gate set to have, but the Eastin-Knill theorem describes a fundamental limitation, in that any gate set we choose capable of approximating arbitrary quantum circuits will contain at least one nontransversal gate.

It is important to emphasize that transversality is not an inherent property of a gate, but rather a relationship between a gate and a particular error correcting code. Thus, we would like to choose the error correcting architecture in such a way that as many commonly used gates as possible are transversal in it.

LIST OF THINGS TO INCLUDE

We need the link between transtransversality (a logical gate is applied by applying the same gate to its constituent physical qubits individually) and being able to manipulate the stabilizers only in order to alter the logical state the patch contains. To introduce this, we need to understand Clifford gates and how they work in the stabilizer formalism.

What is the link between transversality (a logical gate is applied by applying the same gate to its constituent physical qubits individually) and being able to manipulate the stabilizers only in order to alter the logical state the patch contains? By which I mean, why is it that there exists a way for me to mess with the stabilizers and achieve the effect of an X, Y, Z or H gate, but not the T gate? And I know rationally that Eastin-Knill is the answer, since at least one gate is hard, but I don't have a conceptual understanding of why there isn't some manipulation of stabilizers corresponding to the application of the T gate.

ChatGPT's answer: The clean way to see the connection is to separate three layers: (i) the stabilizer group that *defines* the code space, (ii) logical Pauli operators as nontrivial elements of the normalizer of that stabilizer group, and (iii) logical gates as automorphisms of the code space. In the surface (toric) code, the stabilizers are local X-type stars and Z-type plaquettes, and the logical operators are noncontractible X- and Z-strings. A logical unitary is implementable by "manipulating stabilizers" precisely when it corresponds to a symmetry or locality-preserving transformation of this Pauli/stabilizer structure.

Transversality fits naturally into this picture. A transversal gate acts independently on each physical qubit, so conjugation by the global unitary reduces to conjugation of each local Pauli. If the single-qubit gate maps Pauli operators to Pauli operators (as Clifford gates do), then each stabilizer generator—being a product of Paulis—gets mapped to another Pauli product. In other words, the stabilizer group is mapped to itself (possibly with relabeling), and the code space is preserved. That is why Clifford gates are compatible with transversal implementations in CSS codes like the surface code: their defining algebraic property is that they normalize the Pauli group.

When you "manipulate stabilizers" in the surface code—via code deformation, lattice symmetries, boundary twists, or relabeling $X \leftrightarrow Z$—you are again performing operations that map stabilizer generators to stabilizer generators. Conceptually, you are implementing an automorphism of the stabilizer group. Any such automorphism necessarily induces a transformation on the logical Pauli operators, and because the logical Pauli operators generate the logical Pauli group, the induced logical gate must be a Clifford. This is the essential restriction: stabilizer-preserving transformations can only produce logical operations that send Paulis to Paulis.

Now consider the T gate. Conjugation by T does not preserve the Pauli group: for example,

$$TXT^\dagger = \frac{1}{\sqrt{2}}(X+Y),$$

which is not a Pauli operator but a coherent superposition of Paulis. This already tells you something fundamental. If you tried to implement logical T purely by "messing with stabilizers", you would need a transformation that sends a logical Pauli operator to a non-Pauli operator while still mapping the stabilizer group to itself. But stabilizers, by definition, are Pauli products. Any operation that preserves the stabilizer formalism must map Pauli products to Pauli products under conjugation. There is simply no room inside that algebraic structure to accommodate the non-Pauli images produced by T.

From a more geometric/topological perspective, the surface code is a two-dimensional topological stabilizer code. Any locality-preserving logical gate corresponds to a topological symmetry of the anyon model underlying the code. In the surface code, those symmetries form exactly the Clifford group. The T gate would require a transformation that mixes topological charge sectors in a way that cannot be realized by a locality-preserving deformation of the lattice or stabilizers. So it is not just that "at least one gate must be hard" (Eastin–Knill); it is that the entire stabilizer/topological framework only supports Clifford automorphisms.

Conceptually, then, the answer is this: manipulating stabilizers works precisely

4.1.1 Choices to Make When Performing Quantum Computations

dismantle this section entirely! move clifford+T discussion to previous section and move other software/challenges discussion to experiments and analysis part. Then get some of the theory in experiment and analysis and bring it back up here.

There are a few separate challenges with doing operations on quantum data that we must address in order to reach our goal of a fault-tolerant circuit for Grover's algorithm. Assuming we have made some choice of error correcting code, first, ① we must choose an appropriate universal gate set. Next, ② we need to take the operations we want and determine an equivalent calculation using only the gates in our chosen gate set. Finally, we face another problem entirely in taking our chosen gate set to the quantum hardware and ③ figuring out how the circuit we designed will be implemented on the error-correcting patches corresponding to logical qubits.

With regards to ①, *Clifford+T* is a common choice of universal gate set, fittingly comprised of the *Clifford* and *T-gates*. Clifford gates are unitaries that map Pauli operators to Pauli operators under conjugation, which makes them very well-suited as logical gates in LDPC codes like the ones we have seen. Applying Clifford gates to codes based on X and Z stabilizers respects the stabilizer structure and code space, and since they can be implemented by performing single-qubit operations across all qubits, they are also transversal. The possibility of these simple, single-qubit operations not only grants Clifford gates transversality and built-in fault-tolerance, but it also means they are considered computationally easy to implement.

However, the T-gates, given by

$$T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix},$$

are a different story entirely. A requirement for universality, these gates are neither transversal nor easy computationally; owing to the Eastin-Knill restriction, they necessitate complicated workarounds to be implemented fault-tolerantly (which we return to in Section 4.3). For now, Jaques' argument fundamentally relies on the assumption that we will encode the Grover circuit with the Clifford+T gate set, where the number of T-gates is ultimately the determining factor in his estimation of the time complexity of the algorithm. However, if there were a better way to create the circuit that requires less T-gates (or T-gate equivalents), we could possibly see substantial improvements, since the huge estimated cost of implementing T-gates is a major contributor to why Grover's algorithm works out so poorly in practice.

If it becomes at all interesting to explore other options besides the Clifford+T gate set, I will define and discuss them here. Otherwise, I will elaborate a bit and justify my choice of sticking to Clifford+T.

As for the next challenge, while ② is not necessarily easy in isolation, provided we have our operation expressed as a circuit in some other gate set, there fortunately exist tools such as Synthetiq [27] that can do this optimization and search process for us. In our case with Grover's algorithm, the circuit without restricting which gates we can use is well known, and Synthetiq can be used to convert it into whatever other limited gate set we would like.

4.2 Logical Operators in Error Correcting Architecture

In many ways, the trouble with running Grover's algorithm on a quantum machine boils down to ③. While certain logical operators are known or can be identified by the software tools, others might pose more of a challenge.

4.2.1 Implementing Clifford Gates in LDPC Codes

Might add a figure name here

As highlighted earlier in Chapter 2, the gates we need to perform the full circuit for Grover's algorithm are H, Z_f and Z_{OR}. Of these, H is largely considered computationally easy, taking our standard X and Z corrections and swapping them, since H maps $|0\rangle$ to $|+\rangle$, $|1\rangle$ to $|-\rangle$, and vice versa, leaving us in the realm of the computationally easy Clifford gates.

We cannot implement Clifford gates transversally in the 5-qubit code.

4.3 Phase Query Gates in Error Correcting Architecture

The phase query gates Z_f and Z_{OR} are more problematic, however, and will require the computationally hard T-gates.

TBA after I finish coding this and figure out how it works, because this is where the issues with T-gates arise. Will touch upon T-gate implementation in more detail, magic state distillation for instance.

Chapter 5

Jaques' Argument

Not sure whether to include as a separate section or work parts of it into the intro/background/discussion wherever necessary.

In the normal case when we would use Grover's algorithm, we assume no structure. The argument for there being security in this case is that the number of potential keys is so large that we would not be able to do a brute-force search attack on a classic computer in any reasonable amoutn of time (brute-force being the only option here beceause there is no structure to the problem or the way the keys are determined). The circuit for the quantum algorithm Grover's is such that we may have a speedup on the order of the square root. We also know (from my own previous work) that the runtime of Grover's scales with the number of Grover iterates, which themselves scale with the number of queries to the arbitrary funciton f, or the AES algorithm in our case, which is needed to see if a binary string x is a solution. Each Grover iteration requires running the entire AES encryption circuit, and this is the costly part.

There might also be constant factors here we don't know about, so to see the full picture, we need to know exactly how our f, namely AES, will be implemented in a circuit. Jacques provides a table of estimates of different values on page 20 of the PDF (slide 8 in the deck). The three quantities of apparent interest are Clifford gates (simple, computationally easy gates), T gates (a different type of gate that requires a huge number of qubits) and the depth, which I take to mean the number of gates that have to be applied serially, kind of functioning as a proxy for time (as they cannot be run in parallel). Finally, we have an estimate of a couple thousand logical qubits required for each size of AES in the table, but of course there could be millions of qubits hiding behind this number.

We definitely need some form of error correction, since quantum computations are very prone to noise. But we are orders of magnitude off the number of physical qubits we

need in order to be anywhere near the estimated number of necessary logical qubits.

At this point, he introduces NIST's 2017 MAXDEPTH metric as a baseline for how many logical gates/operations the current quantum computing architectures perform serially over certain periods of time, like a year (2^{40}), a decade (2^{64}) or a millenium (2^96)[1]. Additionally, he makes clear that any sort of realistic attack on AES would have to be parallelized.

This project will attempt to tackle just that. What happens if we use something else instead, and what might that be?

[1]He also points out that this limit does not reflect decoherence concerns, i.e. the quantum state collapsing and quantum data disappearing.

Part III

Experiments and Analysis

i'm thinking of a methods/analysis/results structure like this: details on the scaling issues with simulating quantum systems, how they explode unless we find some smart solutions (hint hint stabilizer formalism)
this motivates beginning by a proof of concept experiment with the smallest codes, where i touch upon the 5 and 7 qubit codes and their properties (maybe i don't really introduce them in the theory section at all, leave their formal descriptions as an appendix?). i explain these experiments however far they get with memory limitations etc
moving on to why the stabilizer formalism is a helpful tool that lets us do more classically because we can store a state essentially "in terms of its stabilizers" (right?)
introducing the software package panQEC and some details on how it works internally that support this "easy to compute with stabilizers" argument
then we show the quantum memory simualtion results with the BB code and surface code. now we get into the part i havent really done but i think should be possible: i know panQEC can identify X and Z logicals so it would be interesting to run the simulations again with those applied perhaps, maybe even a hadamard?
then we make a similar plot of the logical operators over time
then we move on to T gates in earnest and CCZ factories and how this is outside the scope of the stabilizer formalism so panQEC probably can't really help us, and then i can show some attempts to code this up on individual logical qubits (not in the grover circuit)

Claude's thoughts: The flow is good — it has a natural narrative logic where each section motivates the next. The stabilizer formalism section in particular is a nice pivot point that earns its place structurally rather than feeling like background dumped in the middle.

On your specific question about stabilizer representation: yes, your intuition is correct. Instead of storing a full 2^n-dimensional state vector, you store the n stabilizer generators as Pauli strings, which scales polynomially. The catch, as you know, is that this only works as long as you stay within the Clifford group — T-gates kick you out of that efficient representation, which is exactly the wall you hit with PanQEC. That's a clean story. On the method/analysis/results distinction

Honestly, I'd stop trying to force these into separate chapters and instead think of it as one unified chapter with a clear internal logic. Here's why: in your structure, the "result" of each experiment is also the "motivation" for the next step. Separating them would actually break the flow you've built. What you could do instead is label sections clearly within a single chapter so the reader always knows what mode they're in:

Sections describing what you did and how = method

Sections showing plots and outcomes = results

Sections interpreting what it means for Jaques' argument = analysis

These can alternate naturally within one chapter rather than being segregated. Many theses in theoretical/computational physics work this way.

On the structure itself, a few concrete thoughts:

The scaling issues section works well as an opener because it frames everything that follows as a response to a constraint. Just make sure it ends with a clear statement of what the stabilizer formalism buys you — that sentence is the hinge the whole chapter turns on. The 5 and 7-qubit codes as proof-of-concept is fine, and putting their formal definitions in an appendix is a sensible call. In the main text you really only need to say: "smallest code correcting all errors, used as a starting point, hit this wall" and "smallest code with transversal Cliffords, used to attempt computation, got this far." The appendix handles anyone who wants the math.

The PanQEC section is where I'd be careful. Explaining how it works internally is only worth including if it directly supports your stabilizer efficiency argument — which it sounds like it does if the code represents states in binary symplectic form. Keep that section tightly focused on "here's why this tool works well for Clifford operations and here's exactly where it stops working," rather than being a general software description.

The X, Z, and Hadamard logical operator simulations sound very doable and would make a genuine contribution — even a plot showing logical error rates with and without active logical operations applied would be something concrete. The key question is whether PanQEC's logical operator identification gives you operators you can actually apply to the encoded state, or just identifies what they are. Do you know which it is?

The T-gate section at the end is where your thesis argument lands, so it needs to be the most carefully written part even if the implementation is the least complete. I'd structure it as: here is what we would need to do (CCZ factory, magic state distillation), here is why the stabilizer framework can't help us simulate it efficiently,

Chapter 6

General Limitations when Simulating Quantum Systems

In order to design and develop the technology necessary to begin implementing new error correcting architectures, it is typically helpful to simulate how the components might work. In particular, the goal of this project is to simulate how a physical qubit setup might look for a working version of Grover's algorithm, a steppingstone on the way to fault-tolerant physical implementation. However, simulating the full fault-tolerant system with an error correcting code of our choice turns out to be nearly impossible, even for the smallest nontrivial case of Grover's algorithm with only two input qubits.

To understand why this is so problematic, recall the expression for the unitary matrix defining an arbitrary quantum circuit with n input qubits:

$$U \in \mathbb{C}^{2^n \times 2^n}.$$

Given a gate set, we obtain U by *lifting* the individual gate matrices to the appropriate dimension $2^n \times 2^n$ and multiplying them in the right order. Specifically, in a quantum circuit, lifting a single-qubit gate $G \in \mathbb{C}^{2\times 2}$ acting on qubit k of an n-qubit system means constructing the corresponding $2^n \times 2^n$ unitary

$$\tilde{G} = \mathbb{I}_2 \otimes \cdots \otimes G \otimes \cdots \otimes \mathbb{I}_2,$$

where G appears in the k-th tensor factor. This approach preserves the mathematical structure of the full state at all times when applying a gate to only a few of the qubits, but it comes at a cost. The exponential scaling when n grows as we add qubits to the system means that we quickly face extremely large matrices when simulating anything beyond minimal examples or the logical abstractions of algorithms. This is bad news for us, because it means that when we add error correction architectures belonging to the families of primary interest, we will not be able to simulate the circuit.

For instance, the smallest possible surface code is displayed in Fig. 6.1. It requires at least ten qubits, provided we reuse a single ancillary qubit for measurement rather than using eight individual ones. Each gate we would need in a simulation of Grover's with two logical qubits would be a $2^{20} \times 2^{20}$ matrix to account for the no fewer than 20 qubits in the system.

Add technical explanation/justification of memory requirements.

This also assumes none of the gates necessitate additional ancilla qubits to further drive up this cost, which, as we will see in Chapter 8 when we delve into the implementation of the T-gate, is not the case if we want a universal gate set.

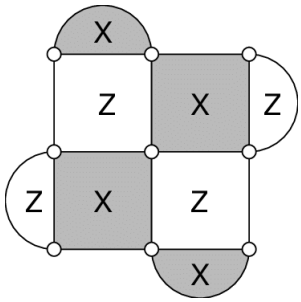


Figure 6.1: The smallest possible surface code consisting of nine data qubits. The indicated eight stabilizers could, in principle, be measured sequentially by a single additional qubit, or measured in parallel by an additional eight qubits. Add source to bibliography.

6.1 Proof-of-concept Experiments with Smaller Codes

I can't find anything but it's scary to declare this. How should I phrase it?

No published fault-tolerant physical implementation of Grover's algorithm exists at any scale, making it worthwhile to explore in detail how that might look. Aware of the limitations that come with simulating a large quantum system classically, two smaller error correcting codes, the *5-qubit code* and the *7-qubit Steane code*, were selected and implemented in code. These codes are defined and described in more detail in Appendix A.

more detail or pseudocode? results?

The 5-qubit code was a natural place to begin when searching for finding ways to simulate Grover's algorithm classically, since it is the smallest quantum error correcting code that can correct any type of error [28]. The code was implemented by explicitly constructing the encoding circuit and logical operators as $2^5 \times 2^5$ unitaries. While the quantum memory aspect worked as intended, it turns out that the 5-qubit code is not a good candidate for computation because in it, the Clifford gates are not transversal. This naturally led to considering the 7-qubit Steane code instead, another relatively small error correcting code where the Clifford gates do possess transversality, simplifying the implementation of certain logical operators. For this reason, the 7-qubit Steane code is a common choice in quantum error correction research settings today.

6.1.1 Simulating Toy Version of Grover's Algorithm

The circuit design for Grover's algorithm without any form for error correction is well-known and is illustrated in Section 2.2.

How was it implemented? Add a diagram and maybe some pseudocode.

Chapter 7

Utilizing Stabilizer Formalism as a Computational Tool

The exponential scaling of full unitary simulation, as described in Chapter 6, presents a fundamental barrier to classical simulation of large quantum circuits. In certain codes, however, we can at least partially circumvent the issue: For an n-qubit system, rather than storing and manipulating the full $2^n \times 2^n$ unitary, or even the state vector of 2^n complex amplitudes, a stabilizer state can be represented compactly by a set of n independent commuting Pauli operators $\{g_1, \ldots, g_n\}$ called *generators*. This uniquely specifies a state $|\psi\rangle$, which is the one joint $+1$ eigenstate of all g_i simultaneously. Thus, to recover the state vector from the generators, one simply asks which state satisfies $g_i |\psi\rangle = +|\psi\rangle$ for all i. To further illustrate how the stabilizer representation lets us recover the corresponding state vector uniquely, we will step through a small example.

Example: Recovering a state vector from the stabilizer representation

Consider the 3-qubit bit-flip code, with stabilizers $g_1 = ZZI$ and $g_2 = IZZ$, and logical operators $\bar{X} = XXX$, $\bar{Z} = ZZZ$. The generators alone define the codespace, spanned by $|\bar{0}\rangle = |000\rangle$ and $|\bar{1}\rangle = |111\rangle$, but do not yet specify which logical state we are in. To pin down a specific state, we augment the generator list with a logical operator. If we add $\bar{Z} = ZZZ$ as a third stabilizer, the unique joint $+1$ eigenstate of all three is $|000\rangle = |\bar{0}\rangle$, since we see that $ZZZ|111\rangle = -|111\rangle$, ruling out $|111\rangle = |\bar{1}\rangle$ as it is not a $+1$ eigenstate of the set. If we add $\bar{X} = XXX$ instead, the unique joint $+1$ eigenstate becomes $\frac{1}{\sqrt{2}}(|000\rangle + |111\rangle) = |\bar{+}\rangle$. In each case, the full state vector is recoverable from the generator list alone, with no ambiguity.

The storage advantage is now clear: Each generator g_i is a tensor product of n single-qubit

Have I previously made an argument for how many stabilizers are required for a given stabilizer code? If not I should do so or expand on this here.

Paulis, requiring only $\mathcal{O}(n)$ bits to store, so the full generator list needs only $\mathcal{O}(n^2)$ bits, compared to $\mathcal{O}(2^n)$ complex numbers for the state vector.

We also have computational shortcuts when working with Clifford gates, which we recall are the gates that map Pauli matrices to Pauli matrices. If we have a stabilizer state $|\psi\rangle$ with generators $\{g_1, \ldots, g_n\}$, we can apply a Clifford gate C to obtain a new state $C|\psi\rangle$. Rather than computing the full matrix product $C|\psi\rangle$ in a 2^n-dimensional space, we can take advantage of the stabilizer formalism and the fact that the generators uniquely determine the state to calculate the effect of C based on what happens to the generators. We want a new set of operators g_i' such that $g_i'(C|\psi\rangle) = C|\psi\rangle$. The following map achieves this:

$$g_i \mapsto Cg_iC^\dagger.$$

We can verify that generators $g_i' = Cg_iC^\dagger$ defined in this way do indeed stabilize our updated state $C|\psi\rangle$, since it holds that:

$$g_i'C|\psi\rangle = Cg_iC^\dagger \cdot C|\psi\rangle = Cg_i|\psi\rangle = C|\psi\rangle.$$

This stabilizer update rule holds for any unitary matrix, but working with Clifford gates unlocks the advantage of ensuring $Cg_iC^\dagger$ is always another valid Pauli string, preserving the compact $\mathcal{O}(n)$-bit-per-generator storage throughout the entire computation. If conjugation by C could produce arbitrary matrices, we would be back to storing exponentially large objects and the efficiency gain would be lost.

To see the mechanics of the update rule, consider applying H to $|0\rangle$, stabilized by $g = Z$. We can simply update the stabilizer $HZH^\dagger = X$ and read off that the new state is stabilized by X, meaning it is the state $|+\rangle$. No matrix multiplication of a state vector is needed; the state update is captured in its entirety by relabeling one Pauli letter. In general, since a Clifford gate touches at most two qubits at a time, updating one generator costs $\mathcal{O}(n)$, relabeling at most two Pauli letters according to a fixed lookup table. Updating all n generators therefore costs $\mathcal{O}(n^2)$ in total, compared to the $\mathcal{O}(2^n)$ cost of full state vector simulation. This is the main result of the Gottesman-Knill theorem [29]: Any circuit composed entirely of Clifford gates can be simulated efficiently on a classical computer in the stabilizer representation.

Have I mentioned this before?

7.1 Software Aids

The software package `PanQEC` [30] was used to visualize the geometry of several error correcting codes and to extract their logical operators. It stores stabilizer codes internally in *binary symplectic format*, in which each Pauli operator on n qubits is represented as a binary vector $v = (v_X \mid v_Z) \in \mathbb{F}_2^{2n}$. Here, the first n bits indicate which qubits carry an

X factor and the last n bits indicate which carry a Z factor. A full stabilizer group with m generators is then stored as an $m \times 2n$ binary matrix, replacing what would otherwise require tracking m dense $2^n \times 2^n$ matrices. This yields a significant reduction in memory overhead and makes operations such as checking commutativity straightforward via the symplectic inner product.

Add explanation of logical operator application in PanQEC.

It also has functionality for determining logical operators of various types, which correct errors and perform small single-qubit computations.

7.2 Simulating Stable Quantum Memory with PanQEC

How are different codes specified? The basics of the software skeleton.

A natural place to begin is finding ways to store quantum data as quantum memory.

Surface code is the most common architecture today. I typically hear we will need 1000 physical qubits per logical qubit with the surface code, which is much larger than we simulate.

source? and write more formally

Liang and Chen [31] have found all BB codes using less than 160 qubits. The smallest of these, the $[[16, 4, 4]]$-code, is the only option I have considered because otherwise I run into major memory issues.

Brekke and Trefjord [23] did a comparison of surface code and BB code as part of a project for NSM, and I essentially recreated that simulation. Write about it and put it into the two next sections.

7.2.1 Results

Explain implementation of quantum memory with figures from PanQEC.

Quantum data storage is one of the key ways BB codes offer an improvement over surface codes.

7.3 Simulating Transversal Logical Operators with PanQEC

The stabilizer formalism can help us make this more efficient for certain gates because we work only with the stabilizers to represent a state. The trouble is when we tackle

the T gate, which is incompatible with our stabilizer space and we have to resort to this clunky notation again with the full matrix. This means that the inlcusion of the T gate prevents an efficient classical simulation.

Explain what we already know about how to do this in real life and how certain logical operators are applied with PanQEC

7.3.1 Results

Do these experiments if possible, recreate summer student plots but with logical operators applied.

Chapter 8

The Trouble with the T-gate

Why can't the stabilizer framework help us simulate it efficiently?

Note that this efficiency breaks down the moment a non-Clifford gate such as the T-gate is introduced. In this instance, a Pauli can then map to a non-Pauli under conjugation, the representation is no longer valid, and one must fall back to full state vector simulation.

As we have seen, applying the T-gate does not in any way reduce to stabilizer operations, it is not immediately obvious how to go from the PanQEC architecture to a working circuit.

8.1 Implemention

What would we need to do to implement t-gates? (CCZ factory, magic state distillation)

Here is what a minimal attempt looks like and where it breaks down

Try to code this up on individual error corrected qubits, not in a full circuit.

Chapter 9

Assessing the Quantum Threat

What does everything we've done so far tell us about Jaques' conclusion?

9.1 T-gate Cost as a Bottleneck

How T-gate count in the Grover circuit translates to physical resource requirements

Comparison across codes: surface code vs. BB code overhead per logical T-gate

Even with BB code's improved qubit ratio, does the T-gate problem go away?

9.2 Improvements with a Better Error Correcting Code

Revisiting Jaques' estimates under BB code assumptions

Where BB code helps (qubit overhead, threshold) and where it doesn't (T-gate implementation still hard)

Provided we need CCZ factories, we still need these in the BB codes. The main differences lie in other aspects. So it's good and bad news in a way, improving CCZ codes will create improvements everywhere, but it doesn't automatically make BB codes better.

Diagrams of logical operator circuits across codes go here

9.3 Grover's Algorithm as a Practical Threat

What would need to change for Grover's to be viable?

Summarize the remaining gaps: T-gate implementation in BB code, circuit depth, parallelization??

Frame this constructively as directions for future work rather than just "it's still hard"

Part IV

Conclusion

Chapter 10

Conclusion

Summarize your findings and future work.

BB codes reduce the overhead everywhere, but the fundamental bottleneck identified by Jaques remains, and closing it requires either better magic state distillation protocols or a gate set that avoids T-gates entirely in favor of something less costly. However, we know that it will be more costly than the Clifford gates in any case.

10.1 Future Work

Does the decoder we use for correcting errors matter at all and what is the efficiency/accuracy tradeoff?

10.2 Declaration of AI Usage

Rewrite this section into formal text.

I have used AI as an aid in coding and will probably continue to do so.
I use it to learn things sometimes and provide intuitive explanations where I otherwise just have equations to study.
I use it very minimally when writing, pretty much just when I can't remember a certain word or if I've used a word a lot and need a synonym (like a thesaurus).
I use it as a search engine when I have complicated search terms that Google can't find for me.
I ask it to format references from websites etc when there is not a downloadable option provided by the authors.

Part V

Appendices

Appendix A

Definitions of Smaller Error Correcting Codes Used

In addition to the aforementioned architectures that are viable options in the landscape of quantum technology today, this project also necessitated some smaller quantum codes to allow for easier simulation of circuitry.

The 5-qubit Code

Explain the 5-qubit code here with relevant mathematical background.

The 7-qubit Steane Code

Explain the 7-qubit code here with relevant mathematical background.

Bibliography

[1] *Symmetric-key algorithms*. Wikipedia. URL: https://en.wikipedia.org/wiki/Symmetric-key_algorithm.

[2] National Institute of Standards et al. *Advanced Encryption Standard (AES)*. en. 2023-05-09 04:05:00 2023. DOI: https://doi.org/10.6028/NIST.FIPS.197-upd1. URL: https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=936594.

[3] Ronald de Wolf. *Quantum Computing: Lecture Notes*. 2023. arXiv: 1907.09415 [quant-ph]. URL: https://arxiv.org/abs/1907.09415.

[4] Samuel Jaques. *Quantum Attacks on AES*. Presentation at the 26th International Conference on Cryptographic Hardware and Embedded Systems (CHES 2024). Available online: https://ches.iacr.org/2024/Jaques_CHES_2024.pdf. July 2024.

[5] Emma Storberg. *TEK5550 Mandatory Assignment 1*. Unpublished midterm exam. University of Oslo, 2025.

[6] Qiskit. *Grover's Algorithm | Understanding Quantum Information & Computation - Lesson 08*. Nov. 2023. URL: https://www.youtube.com/watch?v=hnpjC8WQVrQ.

[7] Wikipedia contributors. *Quantum logic gate — Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Quantum_logic_gate&oldid=1278673600. [Online; accessed 4-March-2025]. 2025.

[8] Wikipedia contributors. *Quantum superposition — Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Quantum_superposition&oldid=1275226563. [Online; accessed 4-March-2025]. 2025.

[9] Huinan Chen et al. *Quantum circuit for implementing AES S-box with low costs*. 2025. arXiv: 2503.06097 [quant-ph]. URL: https://arxiv.org/abs/2503.06097.

[10] Wikipedia contributors. *Big O notation — Wikipedia, The Free Encyclopedia*. [Online; accessed 27-January-2026]. 2026. URL: https://en.wikipedia.org/w/index.php?title=Big_O_notation&oldid=1333898274.

[11] Liao-Liang Jiang et al. "Constructing resource-efficient quantum circuits for AES." In: *Frontiers in Physics* Volume 13 - 2025 (2025). ISSN: 2296-424X. DOI: 10.3389/fphy.2025.1582819. URL: https://www.frontiersin.org/journals/physics/articles/10.3389/fphy.2025.1582819.

[12] Qun Liu et al. *Improved Quantum Circuits for AES: Reducing the Depth and the Number of Qubits*. Cryptology ePrint Archive, Paper 2023/1417. 2023. URL: https://eprint.iacr.org/2023/1417.

[13] Charles H. Bennett et al. "Strengths and Weaknesses of Quantum Computing." In: *SIAM Journal on Computing* 26.5 (Oct. 1997), pp. 1510–1523. ISSN: 1095-7111. DOI: 10.1137/s0097539796300933. URL: http://dx.doi.org/10.1137/S0097539796300933.

[14] Christof Zalka. "Grover's quantum searching algorithm is optimal." In: *Physical Review A* 60.4 (Oct. 1999), pp. 2746–2751. ISSN: 1094-1622. DOI: 10.1103/physreva.60.2746. URL: http://dx.doi.org/10.1103/PhysRevA.60.2746.

[15] IBM Quantum. *Other code families*. Accessed: 2025-01-28. IBM Quantum Learning. 2025. URL: https://quantum.cloud.ibm.com/learning/en/courses/foundations-of-quantum-error-correction/quantum-code-constructions/other-code-families.

[16] A.Yu. Kitaev. "Fault-tolerant quantum computation by anyons." In: *Annals of Physics* 303.1 (Jan. 2003), pp. 2–30. ISSN: 0003-4916. DOI: 10.1016/s0003-4916(02)00018-0. URL: http://dx.doi.org/10.1016/S0003-4916(02)00018-0.

[17] Wikipedia contributors. *Stabilizer code — Wikipedia, The Free Encyclopedia*. [Online; accessed 28-January-2026]. 2026. URL: https://en.wikipedia.org/w/index.php?title=Stabilizer_code&oldid=1332747113.

[18] Arthur Pesah. *The stabilizer trilogy I — Stabilizer codes*. Blog post. Jan. 31, 2023. URL: https://arthurpesah.me/blog/2023-01-31-stabilizer-formalism-1/ (visited on 01/28/2025).

[19] Samira Sayedsalehi et al. "Developing and Analyzing the Defect-Based Surface Codes Using Optimization Algorithms." In: *Quantum Reports* 7.2 (2025). ISSN: 2624-960X. DOI: 10.3390/quantum7020025. URL: https://www.mdpi.com/2624-960X/7/2/25.

[20] Wikipedia contributors. *Pauli matrices — Wikipedia, The Free Encyclopedia*. [Online; accessed 28-January-2026]. 2025. URL: https://en.wikipedia.org/w/index.php?title=Pauli_matrices&oldid=1327001669.

[21] Craig Gidney et al. "Yoked surface codes." In: *Nature Communications* 16.1 (May 2025). ISSN: 2041-1723. DOI: 10.1038/s41467-025-59714-1. URL: http://dx.doi.org/10.1038/s41467-025-59714-1.

[22] Sergey Bravyi et al. "High-threshold and low-overhead fault-tolerant quantum memory." In: *Nature* 627.8005 (Mar. 2024), pp. 778–782. ISSN: 1476-4687. DOI: 10.1038/s41586-024-07107-7. URL: http://dx.doi.org/10.1038/s41586-024-07107-7.

[23] Anton Brekke and Brage A. Trefjord. *Sommerprosjekt 2025*. Summer student project report on GitHub. [Online; accessed 28-Jan-2026]. 2025. URL: https://github.com/Bragit123/QEC/blob/main/rapport.pdf.

[24] Wikipedia contributors. *Tanner graph — Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Tanner_graph&oldid=1296985959. [Online; accessed 28-January-2026]. 2025.

[25] Bryan Eastin and Emanuel Knill. "Restrictions on Transversal Encoded Quantum Gate Sets." In: *Physical Review Letters* 102.11 (Mar. 2009). ISSN: 1079-7114. DOI: 10.1103/physrevlett.102.110502. URL: http://dx.doi.org/10.1103/PhysRevLett.102.110502.

[26] Wikipedia contributors. *Eastin–Knill theorem — Wikipedia, The Free Encyclopedia*. [Online; accessed 26-January-2026]. 2025. URL: https://en.wikipedia.org/w/index.php?title=Eastin%E2%80%93Knill_theorem&oldid=1314440925.

[27] eth-sri. *Synthetiq: Fast and Versatile Quantum Circuit Synthesis*. Code base, GitHub repository. 2025. URL: https://github.com/eth-sri/synthetiq.

[28] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2010.

[29] Daniel Gottesman. *The Heisenberg Representation of Quantum Computers*. 1998. arXiv: quant-ph/9807006 [quant-ph]. URL: https://arxiv.org/abs/quant-ph/9807006.

[30] panqec. *panqec: Simulation and visualization of quantum error correcting codes*. Version main. 2021. URL: https://github.com/panqec/panqec (visited on 02/18/2026).

[31] Zijian Liang and Yu-An Chen. *Self-dual bivariate bicycle codes with transversal Clifford gates*. 2026. arXiv: 2510.05211 [quant-ph]. URL: https://arxiv.org/abs/2510.05211.